System Design — Day 6

Topic: Rate Limiter

1. The Problem and Requirements

A rate limiter caps how many requests a user can make in a given time (e.g. 100 per minute). Request 101 is rejected with HTTP **429 Too Many Requests**. It protects a system from abuse, bots, and accidental overload. Requirements questions to ask first:

- Limit per what: user ID, IP address, or API key?
- Where does it run: inside each app server, or as a separate layer in front?
- Hard limit, or are short bursts allowed?
- Overall scale: how many requests per second?

Note: we need only a running **count** per user, not a stored history of every request.

2. Where the Counter Lives

Store one tiny counter per user in **Redis**, e.g. **user_42** → **37**, with a **TTL of 60 seconds** (Day 3) so it deletes itself and the next minute starts at zero.

Why not in each app server's own memory? That makes servers stateful (Day 1). With 10 servers and traffic spread evenly, each server sees only about 10 of a user's requests, so none reaches its limit and the user gets about 10 × 100 = **1,000** requests through. A shared Redis counter fixes this because every server updates the same number.

Memory check: 10 million users × 100 bytes = 1 GB, which fits in one Redis server.

3. Race Condition and Atomic INCR

If the count is 99 and two servers handle two requests at the same instant, both can read 99, both allow the request, and both write back 100. The user made 101 requests but the count says 100. This is the same read-then-write gap as the Day 5 short code collision.

Fix: Redis **INCR** is **atomic**: it adds 1 and returns the new value as one indivisible step. The two servers get back 100 and 101, and the one that gets 101 rejects the request.

4. Fixed Window Counter and Its Weakness

Time is cut into fixed one-minute windows, each with its own counter that resets at the boundary. **Weakness (boundary burst):** a user sends 100 requests at 12:00:59 and another 100 at 12:01:00, so 200 requests pass in about 2 seconds, double the limit.

5. Better Algorithms

Algorithm	How it works	Trade-off
Sliding window log	Store a timestamp per request in a Redis sorted set. On each request, delete timestamps older than 60 seconds, then count what is left. The window always covers the last 60 seconds from now.	Perfectly accurate but heavy: 10M users × 100 timestamps × 8 bytes = 8 GB
Sliding window counter	Keep two fixed-window counters (current and previous minute) and estimate the rolling count as a weighted blend.	Two small numbers per user, but an approximation that can be slightly off
Token bucket	Each user has a bucket that refills at a steady rate. Each request spends one token; an empty bucket means rejection.	Allows short bursts while enforcing the average rate; widely used by real APIs

Number-system tip: 10 million = 1 crore = 1,00,00,000. 100 million = 10 crore. 1 billion = 100 crore.

6. When Redis Fails: Fail Open vs Fail Closed

	Fail open (AP)	Fail closed (CP)
Behavior	Let all requests through, with no limiting	Reject all requests until Redis is back
Best for	Normal public APIs, where the limiter is a helper and should not take the whole API down	Login/OTP attempts, payments, expensive AI calls, where unlimited traffic is costly or dangerous
Risk	A flood could reach the backend	One failing component blocks every legitimate user

A common middle path for open systems: fall back to a rough local limiter in each server's memory (for example 100 ÷ 10 servers = about 10 per minute each). It is imprecise, but far better than no limit.

7. Key Takeaway

A rate limiter is shared, temporary state, so it belongs in Redis with a TTL. Correctness under concurrency comes from atomic operations, not from check-then-write in application code. Pick the algorithm by trade-off (accuracy vs memory vs burst tolerance), and decide failure behavior by how costly unlimited traffic would be.